Final Project Report
Segmentation in Medical Imaging

ECSE 415: Intro to Computer Vision
04/12/2023

Group 10:
Bruno Marcoux (260869395)
Duranjay Kaundun (260836316)
Yazdan Zinati (260928077)
Patrick Juneau (260928476)
Lakshmi Roy (260925369)



***Abstract*— The goal of this project is to implement a complete pipeline to segment and detect nuclei in medical images. The pipeline will consist of a number of steps: (1) a dataset pipeline (2) an image segmentor , and (2) a detector .**

Keywords*— *Computer Vision, Medical Imaging, DICE, tumor

I. INTRODUCTION

Currently, the detection of tumors in medical images is a time and resource expensive process done by highly skilled medical professionals. However, given that it mostly involves pattern recognition, it is a prime candidate for a computer vision solution. This project aims to implement a complete pipeline to segment and detect nuclei in medical images. The pipeline will consist of a number of steps: (1) a dataset pipeline (2) an image segmentor , and (3) a detector.

II. MAIN BODY

A. *Dataset Pipeline*

The first section of this report aims to prepare a data pipeline so that images, CSV and masks are quickly accessible. Images refer to the medical images of patients. The CSV file contains data such as labels and coordinates of each nuclei. The first channel in each mask image encodes the class labels. The product of the second and third channels encode the unique instance label for each nucleus.

For our pipeline, we assumed that all RGB images, csv files and masks share the same filename. Then we scan all files in the RGB folder and store all the filename(without extension) in an array named `imgs`. We then have functions `rgb(file_name), get_mask(file_name), csv(file_name)` returns the rgb image, mask image and the csv path respectively. File_name refers to `imgs[n]`.

A random image is then loaded from imgs along with its respective mask and csv file. We then extract cords_x, cords_y from csv for each raw classification using csv2.reader() and then use cv2.polylines to draw the shapes as shown in Figure 1.

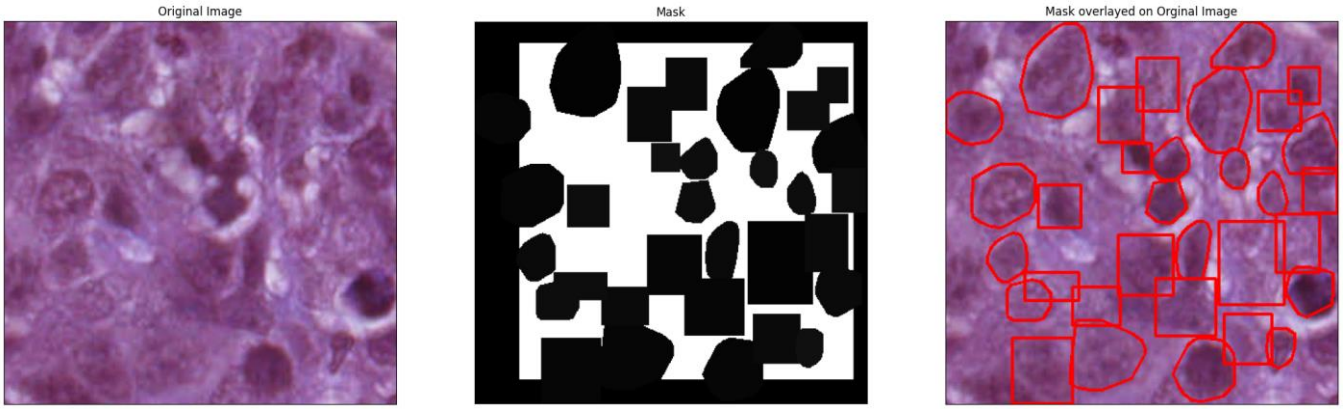


Figure 1: Mask Overlay

The array of images had then to be compressed with only unique IDs. First we loop through all the filenames in imgs and the IDs of each file is extracted using "`id = str.split("_id")[0]`" and saved in a list. To only keep the unique IDs, the list was converted to a set and converted to a list using the following code "`ids = list(set(ids))`". We then created a training and validation split with the ratio 80:20 respectively. We obtained 174 unique patients from 1744 samples.

The aim of the following sections is to be able to segment nuclei groups from their background and then compare it to the ground truth provided. This report will investigate using both the unsupervised and the supervised method.

B. *Segmentation - Unsupervised*

For the unsupervised method, we use K-means clustering to segment the input image and then attempt to isolate the different nuclei groups. For the input parameters of the K-means method from cv2, the termination criteria was set to `cv2.KMEANS_PP_CENTERS` so we get consistent centers at every run.

To find the optimal value of K, the dice coefficient was compared between segmented images using different K. Dice coefficients were evaluated using the segmented mask from K-means and the ground truth mask using the following formula.

$$D = \frac{2 * |X \cap Y|}{|X| + |Y|}$$

We iterated through multiple values of K and calculated their respective dice coefficients. A graph of dice coefficients against K was plotted as shown in Figure 2. The value of K producing the maximum value for the dice coefficient was extracted.

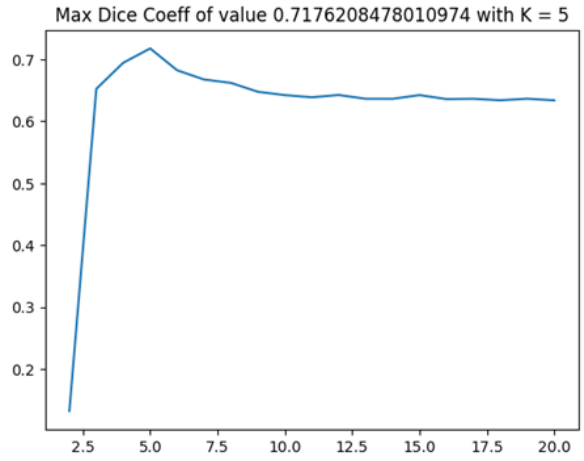


Figure 2: Dice vs K

This process was iterated through all the RGB test images and a graph of frequency against K was plotted as shown in Figure 3. The value of K with the highest frequency will have the highest probability to have a better segmentation mask. As we can observe from the plot, K=5 has the highest frequency and hence this value will be used in the following sections.

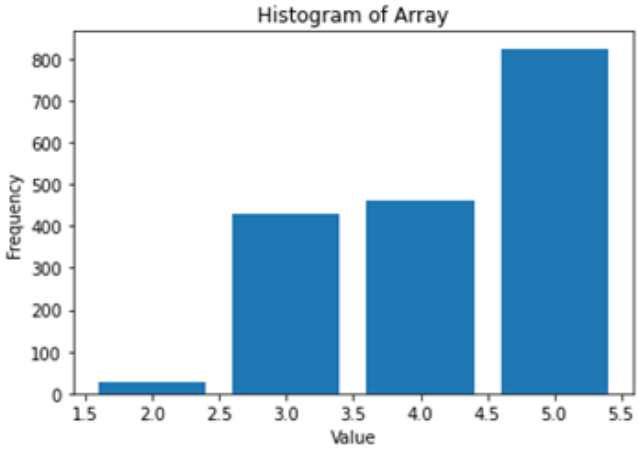


Figure 3: Frequency of K

For sample 682, we obtain the following dice coefficients as shown in Figure 4. As we can observe from the image, K=5 produces a better mask.

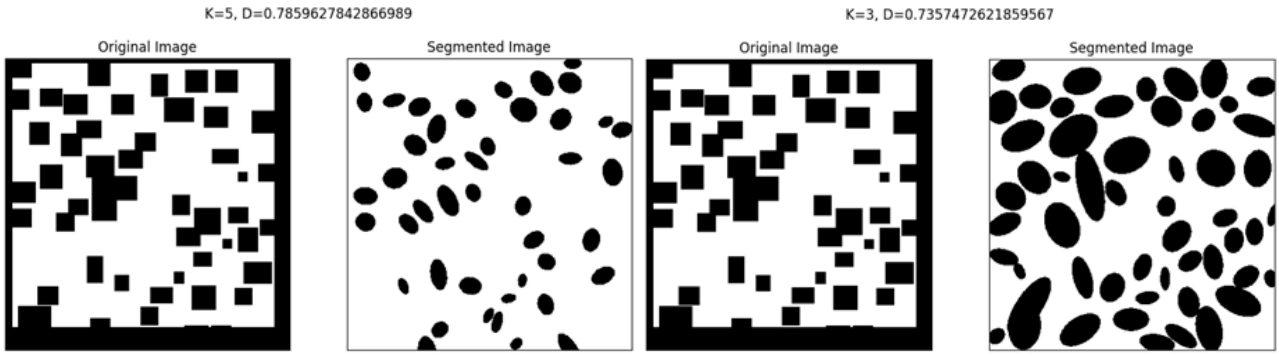


Figure 4: Mask Comparison

After K-means clustering, the result image is converted to grayscale. A binary mask is created with the darkest pixels detected in the image. Contours are then detected from the binary mask. Cv2.fiteclipse is then used to approximate ellipses from contours. The process is shown in Figure 5.

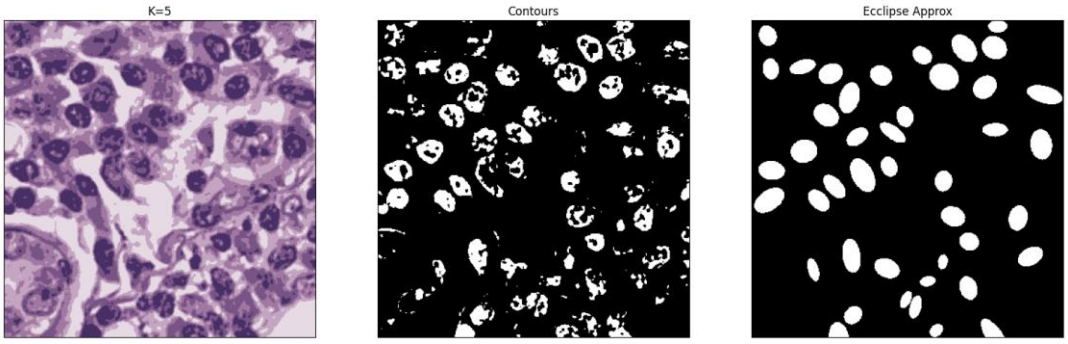


Figure 5: Ellipse Approximation

The overlay of the resultant mask is then compared against the ground truth for K=3 and K=5 as shown in Figure 6.

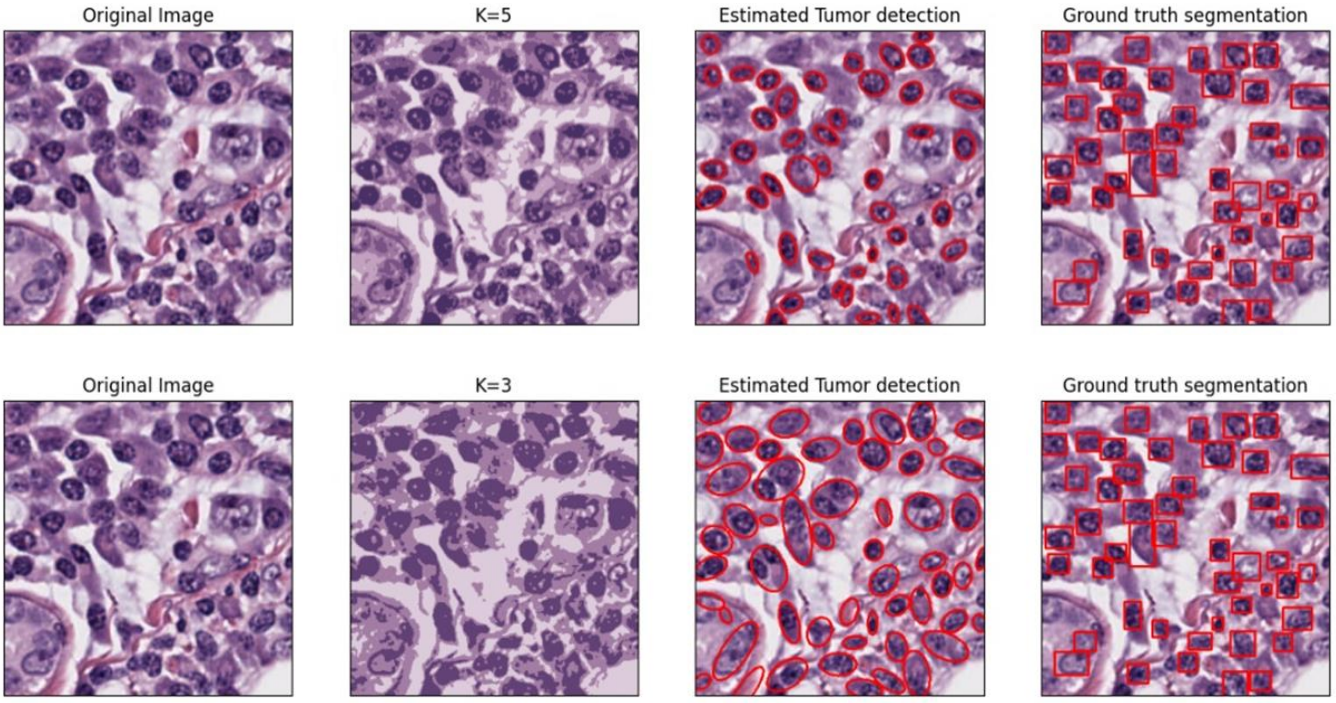


Figure 6: Mask vs Ground Truth

If we repeat the same process with grayscale images and we plot the frequency of K against K we obtain the following plot shown in Figure 7.

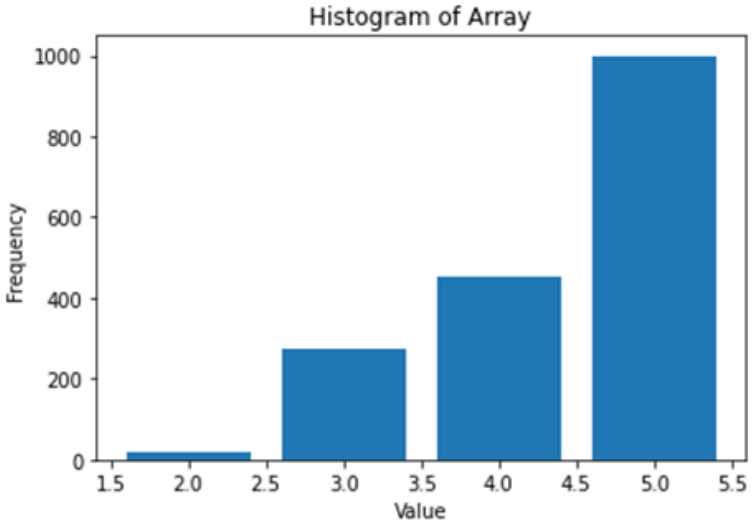
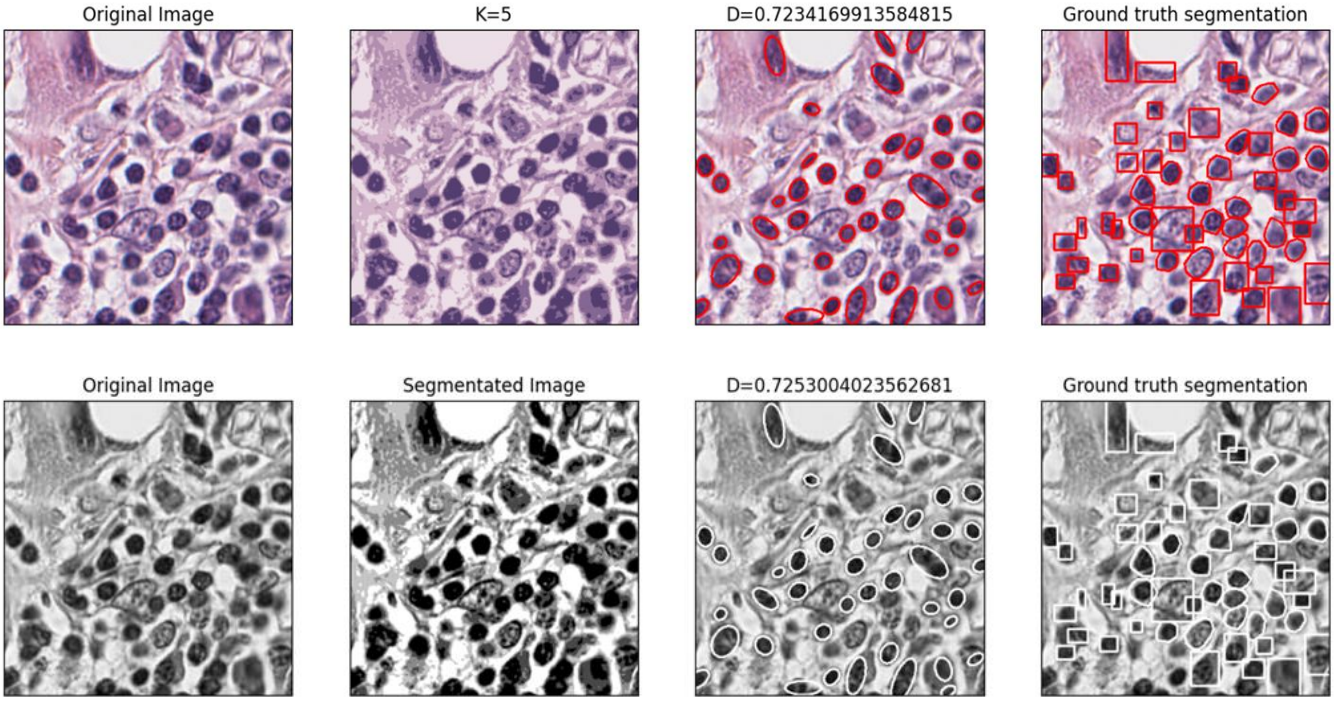


Figure 7: Frequency of K

As we can observe from the plot, K=5 is still dominant to obtain better masks. However, if we compare the results to the RGB plot, we can see that K=5 for grayscale seems to have a higher frequency. That means we will obtain better results for more images if we use grayscale and K=5. This can be concluded in Figure 8 and Table 1.



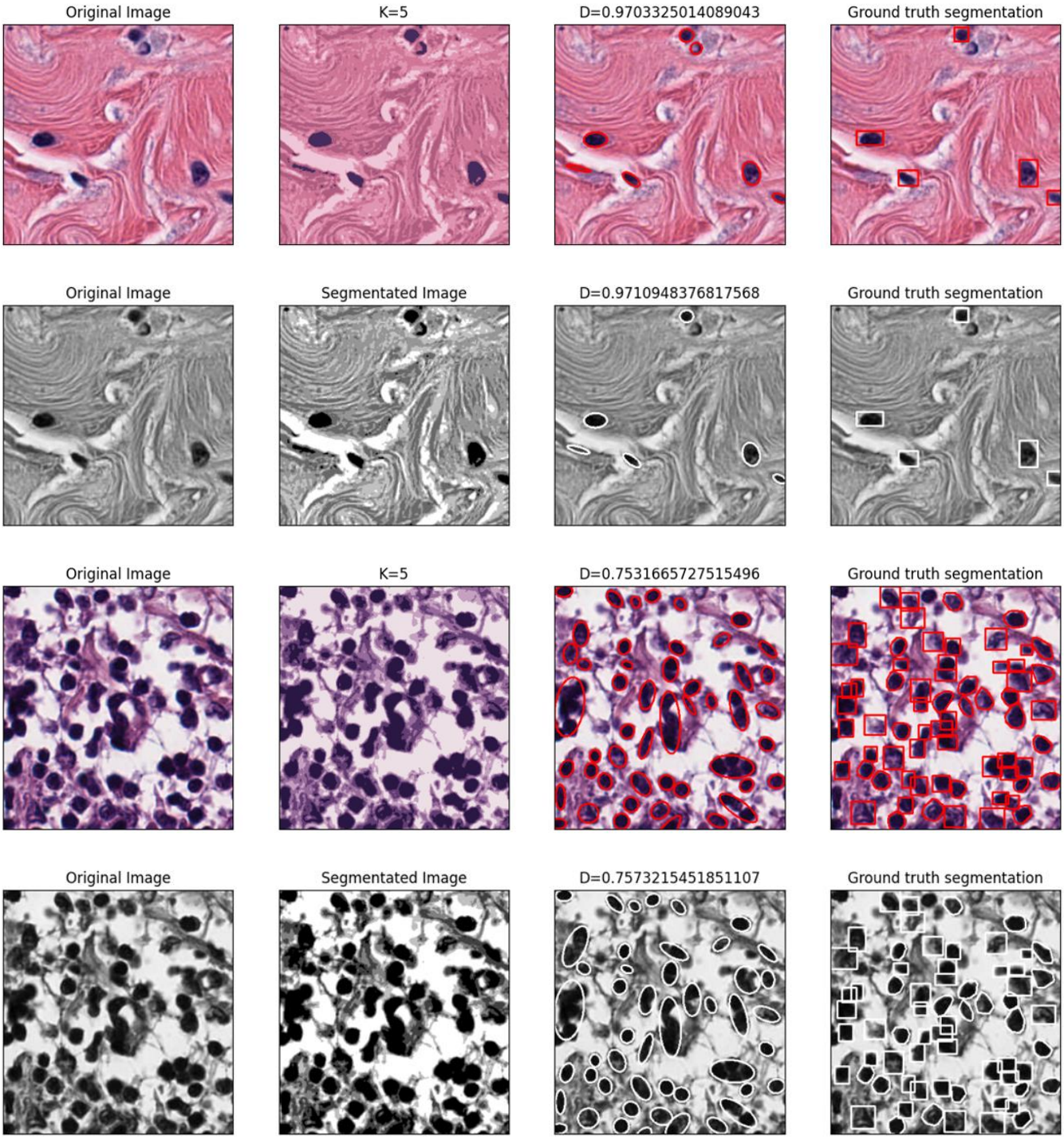


Figure 8: D for Grayscale and RGB

Images	D_RGB	D_grayscale
1	0.7234	0.7253
2	0.9703	0.9711
3	0.7532	0.7573

Table 1: D for Grayscale and RGB summary

Our unsupervised segmentation did manage to detect most nuclei. However, some improvements are listed in Figure 9.

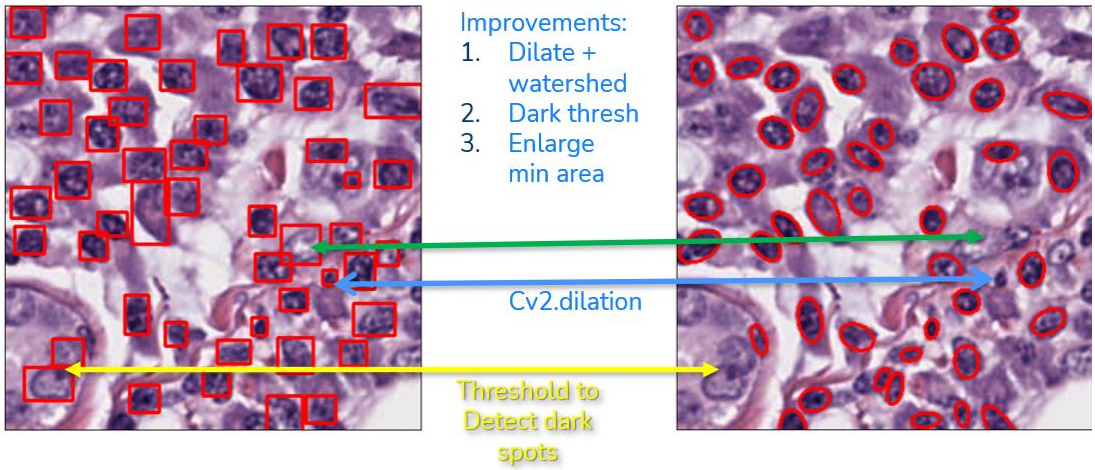


Figure 9: Improvements

The first improvement can be to dilate the ellipses. If we compare the ellipses in Figure 4, we can observe that for K=5, the ellipses tend to be smaller, but they are detected better. Therefore we can use cv2.dilate. This will keep the center of all ellipses the same and increase the size by a percentage. The second improvement involves changing the 'dark' threshold in our pipeline. As mentioned above, we create the mask by extracting the pixels with the darkest color. This can be changed by adding a threshold. This will allow more weak spots to be detected. For the third improvement, we will enlarge the minimum area for the eclipse to be considered a nucleus. After some tweaking, smaller nuclei should be able to be detected.

K-means was also compared to another unsupervised clustering technique: GMM. The results are shown in Figure 10. As we can observe, for K-means we get a dice coefficient of 0.789 while for GMm a dice coefficient of 0.785 is obtained, concluding that K-means works better for our algorithm.

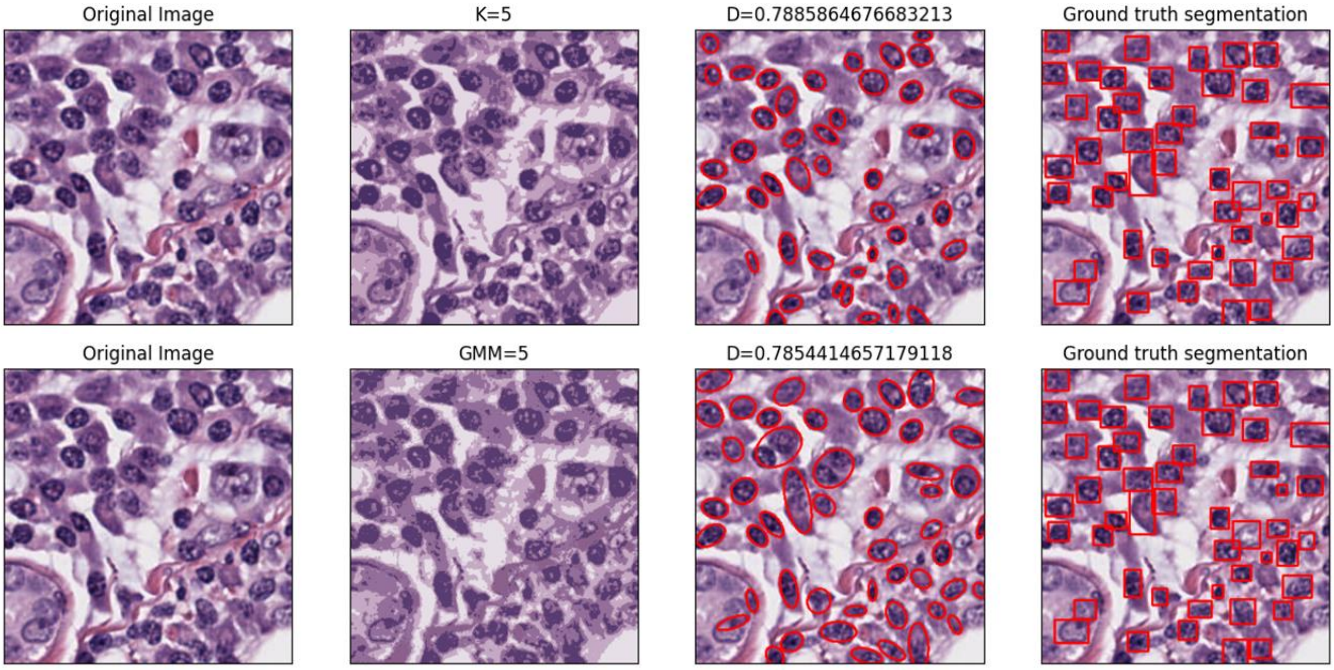


Figure 10: GMM vs K-means

C. *Segmentation - Supervised*

Given that the mask for different nuclei groups are available, each pixel for all images can be labelled. To achieve this, the training images are first converted to grayscale. Then, using the tumor region coordinates in the CSV files, the pixel-wise labels can be determined. Supervised learning segmentation can then be performed by casting the problem into a supervised classification framework.

4.2.1) Describe the supervised approach you are using

When deciding on the supervised learning classification method to use, two options were shortlisted : Support Vector Machine (SVM) and Random Forests. The pros and cons of both solutions are shown in Figure 11.

Support Vector Machine	Random Forests
Pros: • Works well with small and medium sized datasets • Better when features < samples	Pros: • Works well with all sized datasets • Can work with features >> samples
Cons: • Computationally expensive w/ big datasets • Sensitive to outliers and noise	Cons: • Prone to overfitting if too deep • Hyperparameter tuning can be complex

Figure 11: SVM vs. RF

Given the reasons established above, we settled on RF as this classification algorithm it is better suited for large datasets and is better at dealing with noise and outliers.

4.2.2) Report the performance of your supervised segmentor on training images

The obtained confusion matrix for training images is shown in Figure 12.

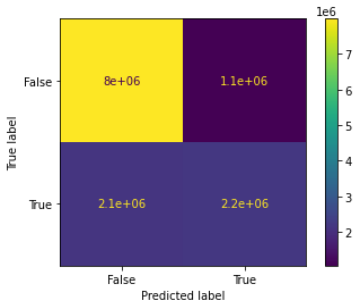


Figure 12: RF performance on training data

The overall accuracy obtained on training images was : 0.761614

4.2.3) Report the performance of your supervised segmentor on validation images

The obtained confusion matrix for validation images is shown in Figure 13.

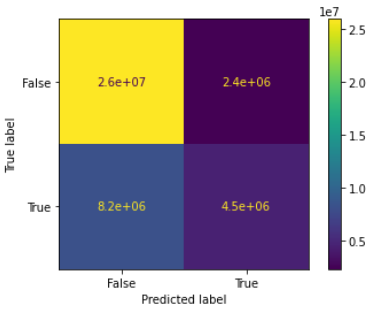


Figure 13: RF performance on validation data

The overall accuracy obtained on validation images was : 0.742694

Therefore, we can see that we successfully mitigated most of the overfitting as our validation results are in line with our training results. However, ~ 75% accuracy shows that there is still room for improvement. The main limitation faced is the mediocre input data. Indeed, the provided ground truth labels are often brute generalizations: square boxes around round tumors. In future works, improving the input data should yield a more accurate model.

4.2.4) Select 3 samples from the validation images and compare the performance of the supervised and unsupervised methods

I. Qualitatively (first line is unsupervised, second is supervised:

Sample 1016:

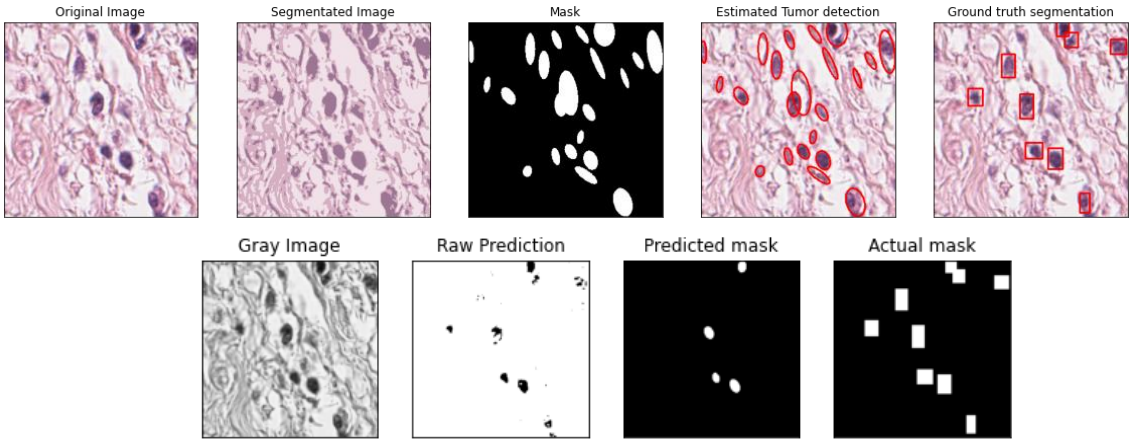


Figure 14: Unsupervised vs Supervised on sample 1016

Sample 632:

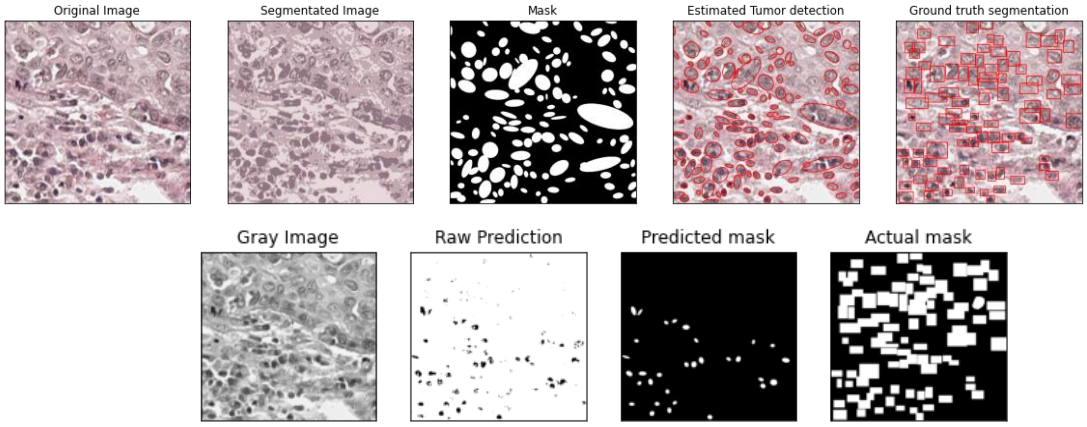


Figure 14: Unsupervised vs Supervised on sample 632

Sample 787

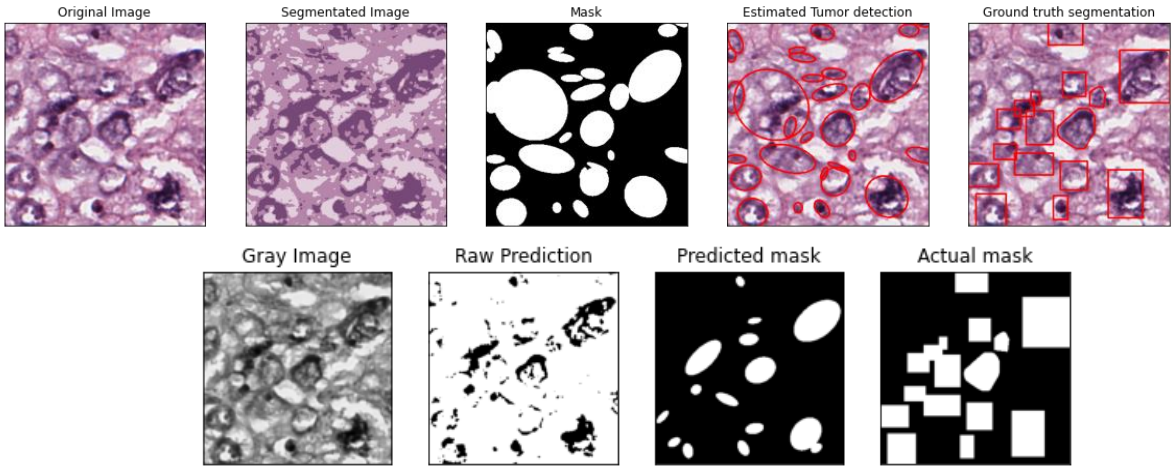


Figure 16: Unsupervised vs Supervised on sample 787

From this qualitative evaluation, it is difficult to draw conclusions on which method leads to better results. Indeed, both methods have mistakes and have difficulties with different scenarios. From sample 632 we can see how the supervised method has difficulty distinguishing the tumors in the upper right side of the image as these show less contrast. On the other hand, from sample 787, we can see the unsupervised method's tendency to group smaller groups of tumors into one big tumor blob.

II. Quantitatively:

Sample	Unsupervised Accuracy	Supervised Accuracy
1016	0.9047604920691429	0.9489339820418496
632	0.746181878877236	0.6826220348211057
787	0.7603747464666302	0.7511912044042368
1008	0.8476073284112661	0.9286482544891076
781	0.7095524123231932	0.7464980566921595
1334	0.7874036590112884	0.7881198910081744
1339	0.6949054288233952	0.70546662719777
855	0.7169017799225873	0.7539401370048543
128	0.7570666197513488	0.6918836500117288
AVERAGE	0.7694	0.7774

From the quantitative analysis, it is shown that the supervised method leads to better results, however only by ~1%

All in all, the supervised method led to better results overall. This is clearly shown in the quantitative analysis. However, even within this limited test set, outliers are present. Therefore, the next step would be to perform this analysis on a bigger batch in order to have more statistically meaningful results.

4.2.5) Which nuclei group can be segmented easily? Discuss by showing some sample visualizations

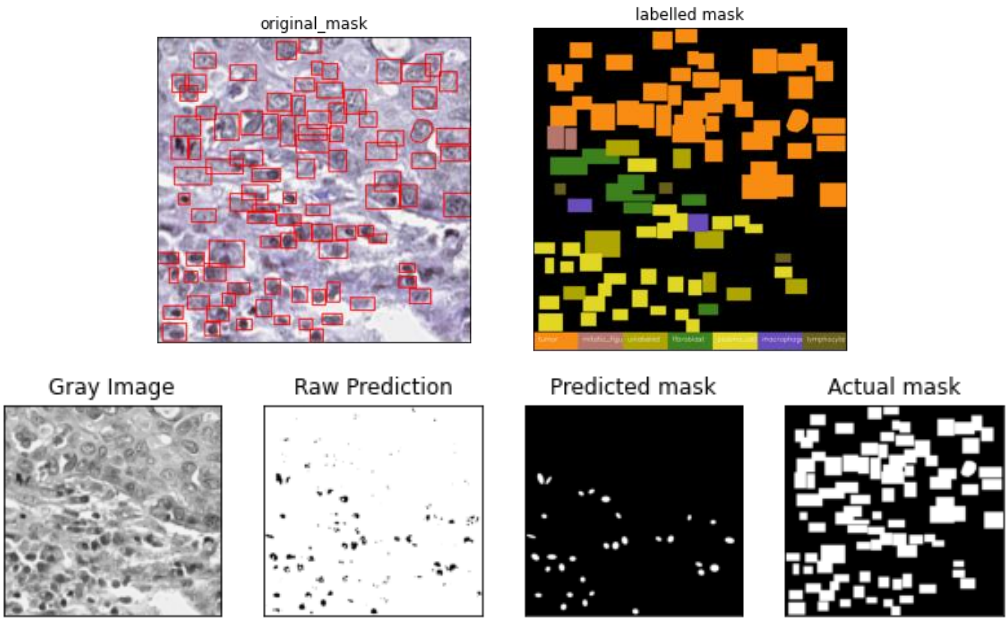


Figure 17: Truth Labels and Found tumors for sample 632

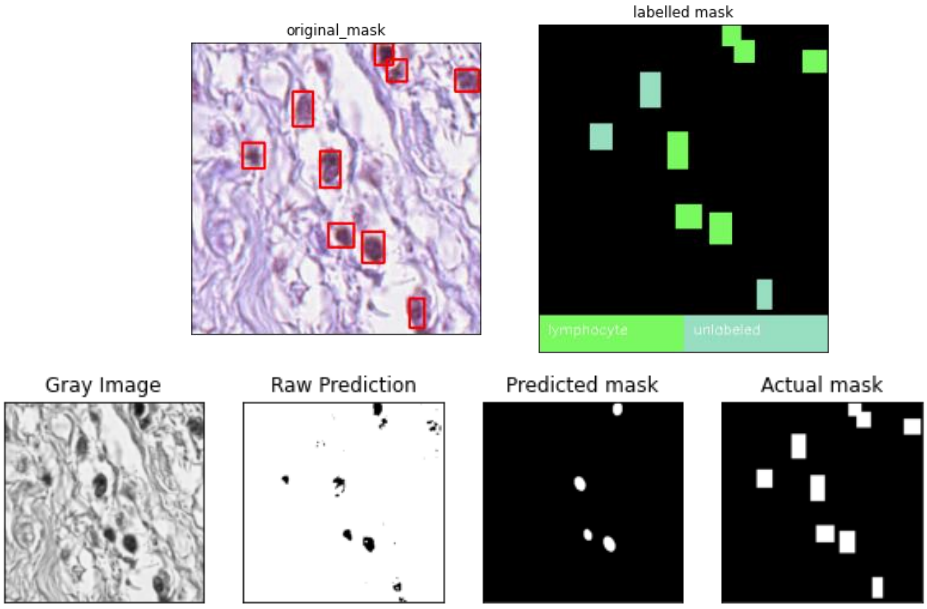


Figure 18: Truth Labels and Found tumors for sample 1016

From the first example, sample 632 in Figure 17, we can clearly see that most of the points of interest in the upper right corner of the images are not segmented properly. When cross-comparing with the ground truth labels, we see that these are mostly labeled as 'Tumors'.

On the other hand, the nuclei groups that are segmented the easiest are ‘Monoblasts’ and ‘Lymphocytes’. Indeed in both Figure 17 and Figure 18 we can see that these are clearly segmented.

4.2.6) Why do you think the nuclei you mentioned in the previous question are easy to segment?

From the above examples, it becomes evident that the supervised random forest classifier works best when the nuclei have sharp edges and high contrast when compared to surrounding healthy regions. This is the case for the Monoblasts in sample 632 which indeed have sharp edges and high contrast.

D. Detecting and Counting Tumors

5.1) *Data Preparation*

We perform a set of preprocessing tasks to prepare the dataset for detecting tumors from pathological blobs and counting tumors in an image. We start by converting all images and masks in the dataset to greyscale. This conversion has many reasons, as it can reduce complexity and help reduce noise in images (Figure 19 - Step 1). Next, we multiply the image and its corresponding mask element-wise to prepare for extracting regions of interest corresponding to diverse pathological structures and tumors. An example image from this preprocessing step can be seen in step 2 of Figure 19. It can be seen that although regions of interest are separated from the rest of the image, they are separated by what looks like a dense salt and pepper noise. We thus remove this background noise and replace those areas with black pixels using a binary mask (see Figure 19 - step 3 for the resulting image).

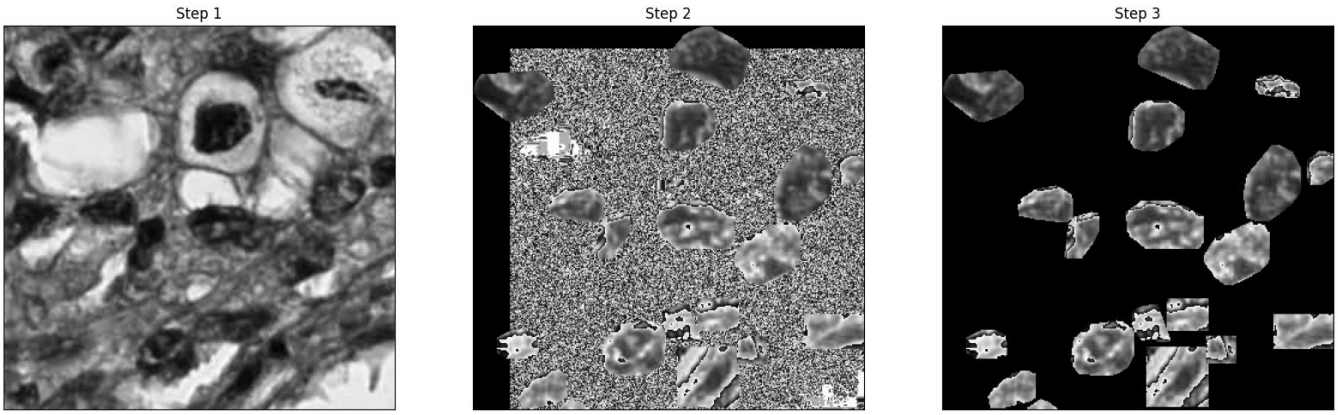


Figure 19: Preprocessing steps for the tumor detection task.

Next, using the coordinates given in CSV files, we extract individual blobs for each image by contouring them and extracting them into separate images. Furthermore, in this step, we use given annotations for respective regions of interest blobs to create a list of ground truth labels for training (See Figure 21). Figure 20 shows an example of three blobs extracted into individual images. We then run Scale-invariant feature transform (SIFT) on these new images to extract key points and descriptors. Extracted descriptors are then used to create feature vectors for training. We offer both binary label vectors (tumor/nontumor) and labels containing the full structure names. The last step consists of slitting the dataset into train and validation sets based on train and validation patients recorded previously.

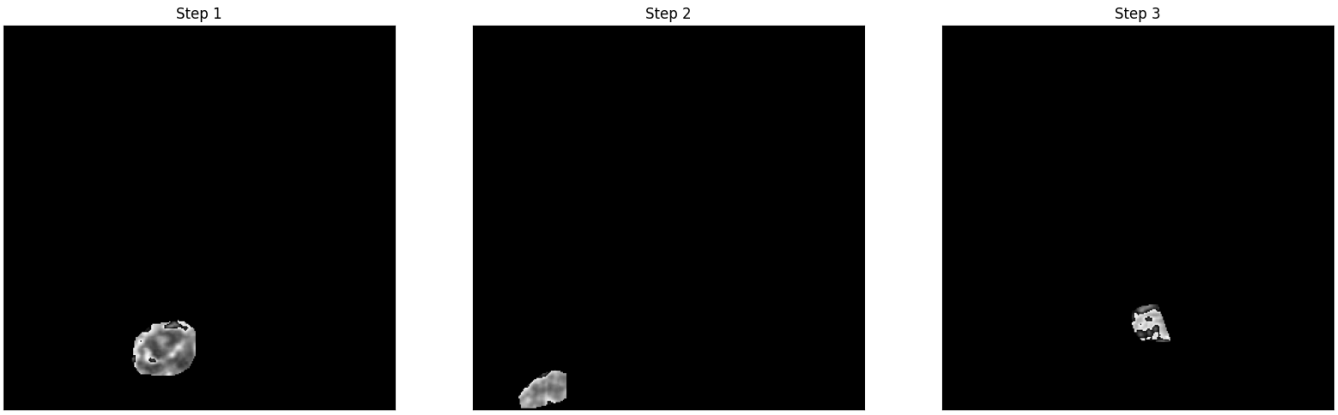


Figure 20: Extracted blobs into separate images

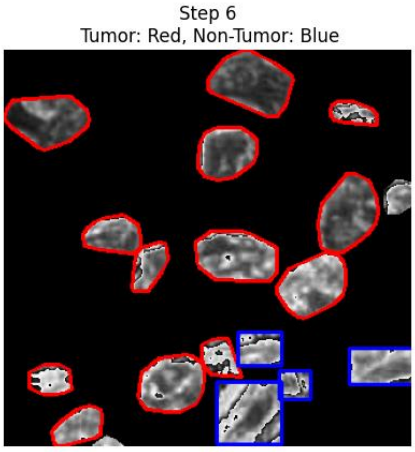


Figure 21: labeled regions on interest corresponding to Figure 19 - Step 3

5.2) *Classification Framework*

5.2.1) In the first step, in order to report performance metrics of the detection model for training and validation images, we shrink the training set and observe the accuracy when using varying ranges of X_train sizes. We observe that 5000:6000 worsens the accuracy, so we vary the numbers around this range. We also performed tests where the SVM kernel is set in rbf mode and it always outperforms the poly mode as well as the linear mode by a large amount. Also, in our tests, we observed that a larger C value brought higher accuracy since it allows the hyperplane to misclassify less data. We also tried experimenting with the value of g; however, we found that the default value performed best. Using the SVM, we then train the classifier, make predictions on the validation data, and calculate the accuracy of the SVM classifier. Figure 22 below is the classification report and accuracy score from the findings.

	precision	recall	f1-score	support
nontumor	0.75	0.80	0.77	2509
tumor	0.64	0.59	0.61	1580
accuracy			0.72	4089
macro avg	0.70	0.69	0.69	4089
weighted avg	0.71	0.72	0.71	4089

Accuracy: 71.53%

Figure 22: Performance Metrics of Validation Data

We repeat the SVM training steps on the training data, giving the performance metric results in Figure 23 below:

	precision	recall	f1-score	support
nontumor	0.93	0.95	0.94	9850
tumor	0.93	0.91	0.92	7274
accuracy			0.93	17124
macro avg	0.93	0.93	0.93	17124
weighted avg	0.93	0.93	0.93	17124

Accuracy: 93.33%

Figure 23: Performance Metrics of Training Data

5.2.2) In the second step, we count the number of detector tumors in the images. To count the tumors, we loop over all the predicted tumors by adding all the blobs that are labeled as tumors for a given image, and store this in an array. The RMSE for the training set is determined to be approximately 0.0667 and the RMSE for the validation set is approximately 0.4715.

5.2.3) In the third step, we view the difference in performance when using the original, unmasked image versus using a mask. To observe the performance of unmasked images, we recreate Xtrain and compute the SIFT keypoints, flatten the array to fit SVM, and then train the SVM classifier. When calculating the accuracy of the new SVM classifier on the training data, we observe the following metrics in Figure 24 below.

	precision	recall	f1-score	support
nontumor	0.93	0.95	0.94	9850
tumor	0.93	0.91	0.92	7274
accuracy			0.93	17124
macro avg	0.93	0.93	0.93	17124
weighted avg	0.93	0.93	0.93	17124

Accuracy: 41.97%

Figure 24: Performance Metrics of Unmasked Training Data

We can see that when using the original, unmasked image, the accuracy decreased greatly (93.33% accuracy when using a masked training image, versus 41.97% accuracy when using unmasked training images). Similarly, the accuracy decreases from 71.53% to 38.89% when using an unmasked validation image. This may be due to increased noise in the image without the mask, making it more difficult for the model to identify pathological regions, but also is affected by the fact that these measurements are taken based on the number of tumors in a given image and not just singular blobs. This means that as the number of blobs goes up for a given image, the probability of the predictor getting the wrong total number of tumors increases.

5.2.4) In this step, we train the SVM with all individual cell types instead of classifying cells as “tumor” and “non tumor” so that we are able to see which cell types are best predicted and which are most often confused with other types. It is worth mentioning that when predicting using the validation set on this new training data, we only get an accuracy of 46.81% since the cell types are often confused for others now.To determine how prevalent this is, we plot the confusion matrix of all the predicted cell types as shown in Figure.. By looking at the results, we are able to determine that our model often mis-predicts fibroblasts as tumors making it harder to tell apart from tumors. We can also see that eosinophils are the easiest cell types to predict; however, this might be due to the small sample size of this cell type used in our validation data.

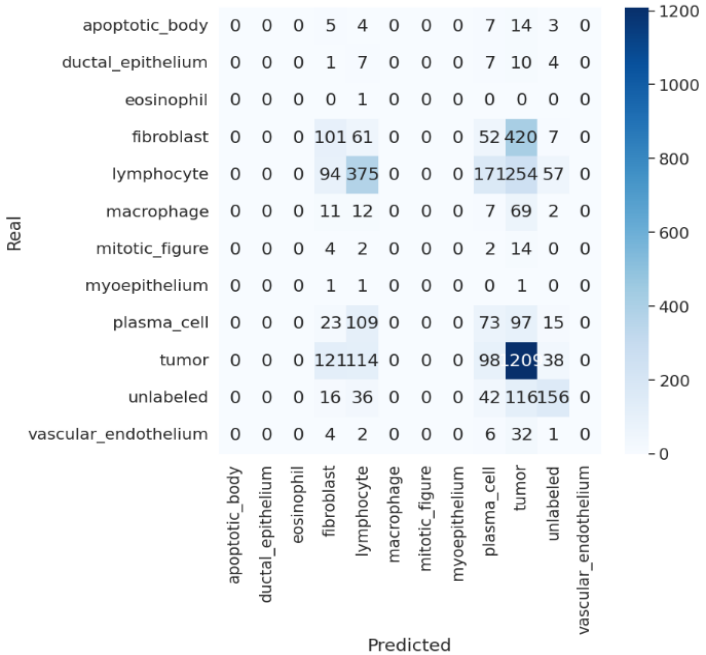


Figure 24: Confusion Matrix

III. CONCLUSION

A complete pipeline to segment and detect nuclei in medical images was described, created and tested in this project. We demonstrated that this task can be automated.